

to `len()`.

This list contains the big O orders of some common Python operations for mapping types, such as dictionaries, sets, and frozensets:

`m[key]` reading and `m[key] = value` assignment $O(1)$ operations.

`m.add(value)` An $O(1)$ operation.

`value in m` An $O(1)$ operation for dictionaries, which is much faster than using `in` with sequences.

`for key in m:` An $O(n)$ operation.

`len(m)` An $O(1)$ operation, because Python keeps track of how many items are in a mapping, so it doesn't need to recount them when passed to `len()`.

Although a list generally has to search through its items from start to finish, dictionaries use the key to calculate the address, and the time needed to look up the key's value remains constant. This calculation is called a HASHING ALGORITHM, and the address is called a HASH. Hashing is beyond the scope of this book, but it's the reason so many of the mapping operations are $O(1)$ constant time. Sets also use hashing, because sets are essentially dictionaries with keys only instead of key-value pairs.

But keep in mind that converting a list to a set is an $O(n)$ operation, so you don't achieve any benefit by converting a list to a set and then accessing the items in the set.

Analyzing Big O at a Glance

Once you've become familiar with performing big O analysis, you usually won't need to run through each of the steps. After a while you'll be able to just look for some telltale features in the code to quickly determine the big O order.

Keeping in mind that n is the size of the data the code operates on, here are some general rules you can use:

- If the code doesn't access any of the data, it's $O(1)$.